



Revisão Acelerada de Programação - RAP RAP-2026, Marinha do Brasil

Centro de Instrução Almirante Alexandrino

Departamento de Aperfeiçoamento Avançado para Oficiais

Curso de Aperfeiçoamento Avançado em Guerra Acústica (C-ApA-Of-GA)

Professor: João Victor da Fonseca Pinto

Descrição do Documento: Lista de Exercícios

Código da Aula: 5

Leia as instruções antes de fazer a lista:

Não é necessário testar se os dados passados por argumento são válidos a menos que pedido na questão.

Q1. Escreva uma função em Python que busca todas as ocorrências de um determinado número em uma lista.

- a) A função deve receber por argumento uma lista de números e um número (nessa ordem), e retornar uma nova lista contendo as posições (índices) em que esse número aparece na lista passada por argumento. Se o número não existir na lista, a função deve retornar uma lista vazia.
- b) Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** `busca_posicoes([10, 20, 30, 20, 40, 20], 20)` → **Saída Esperada:** `[1, 3, 5]`
- **Entrada:** `busca_posicoes([1, 2, 3, 4, 5], 99)` → **Saída Esperada:** `[]`
- **Entrada:** `busca_posicoes([], 5)` → **Saída Esperada:** `[]`

Resposta Questão 1:

```
def busca_posicoes(lista, numero):  
    posicoes = []  
    i = 0  
    while i < len(lista):  
        if lista[i] == numero:  
            posicoes.append(i)  
        i += 1  
    return posicoes
```

Testes (Item b):

```
print(busca_posicoes([10, 20, 30, 20, 40, 20], 20)) # Esperado: [1, 3, 5]  
print(busca_posicoes([1, 2, 3, 4, 5], 99))           # Esperado: []  
print(busca_posicoes([], 5))                         # Esperado: []
```

Q2. O produto escalar entre dois vetores a e b de tamanho n é definido por:

$$[a_1, a_2, \dots, a_n] \cdot \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} = a_1 \cdot b_1 + a_2 \cdot b_2 + \dots + a_n \cdot b_n$$

- a) Escreva uma função em Python que recebe por argumento duas listas que representam vetores e retorna o resultado do produto escalar entre elas. Considere que as listas recebidas possuem o mesmo tamanho e que todos os seus elementos são números.
- b) Teste a sua função de acordo com as entradas e saídas esperadas a seguir:
- **Entrada:** `produto_escalar([1, 2, 3], [4, 5, 6])` → **Saída Esperada:** `32`
 - **Entrada:** `produto_escalar([2.5, 0.0], [4.0, 10.0])` → **Saída Esperada:** `10.0`
 - **Entrada:** `produto_escalar([], [])` → **Saída Esperada:** `0`

Resposta Questão 2:

```
def produto_escalar(a, b):  
    resultado = 0.0  
    if any(isinstance(x, float) for x in a + b) else 0  
    i = 0  
    while i < len(a):  
        resultado += a[i] * b[i]  
        i += 1  
    return resultado
```

Testes (Item b):

```
print(produto_escalar([1, 2, 3], [4, 5, 6]))           # Esperado: 32  
print(produto_escalar([2.5, 0.0], [4.0, 10.0]))       # Esperado: 10.0  
print(produto_escalar([], []))                         # Esperado: 0
```

Q3. Dada uma lista de elementos que representam os pontos de uma função $f(t)$ em um intervalo de tempo de interesse, separados por um passo Δt fixo, uma forma de calcular a área sob a curva $f(t)$ neste intervalo é o método dos trapézios, dado por:

$$\text{área} = \sum_{x=1}^n \frac{[f(x) + f(x-1)]\Delta t}{2}$$

onde x representa a posição na lista e n é igual ao tamanho da lista menos 1.

- a) Escreva uma função em Python que recebe dois argumentos: uma lista de números que representam uma função $f(t)$ e o valor do passo Δt , e retorna a área calculada de acordo com a expressão acima.
- b) Teste a sua função de acordo com as entradas e saídas esperadas a seguir:
- **Entrada:** `area_trapezio([2, 4, 6, 8], 1.0)` → **Saída Esperada:** `15.0`

- **Entrada:** `area_trapezio([1, 1, 1, 1], 0.5)` → **Saída Esperada:** `1.5`

Resposta Questão 3:

```
def area_trapezio(f, delta_t):  
    if len(f) <= 1:  
        return 0.0  
  
    area = 0.0  
    x = 1  
    while x < len(f):  
        area += ((f[x] + f[x - 1]) * delta_t) / 2.0  
        x += 1  
    return area
```

Testes (Item b):

```
print(area_trapezio([2, 4, 6, 8], 1.0)) # Esperado: 15.0  
print(area_trapezio([1, 1, 1, 1], 0.5)) # Esperado: 1.5
```

Q4. Escreva uma função em Python que calcula a diferença entre duas listas.

- a) A função deve receber duas listas por argumento e retornar uma nova lista que contém somente os números da primeira lista que não estão contidos na segunda lista. **Atenção:** Não utilize o operador `in`!
- b) Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** `diferenca_listas([1, 2, 3, 4, 5], [2, 4, 6])` → **Saída Esperada:** `[1, 3, 5]`
- **Entrada:** `diferenca_listas([10, 20], [10, 20, 30])` → **Saída Esperada:** `[]`
- **Entrada:** `diferenca_listas([1, 2, 3], [])` → **Saída Esperada:** `[1, 2, 3]`

Resposta Questão 4:

```
def diferenca_listas(lista1, lista2):  
    resultado = []  
    i = 0  
    while i < len(lista1):  
        elemento = lista1[i]  
        # Verifica se 'elemento' está na lista2 sem usar o operador 'in'  
        encontrado = False  
        j = 0  
        while j < len(lista2):  
            if lista2[j] == elemento:  
                encontrado = True  
            j += 1  
  
        if not encontrado:
```

```

        resultado.append(elemento)
    i += 1
    return resultado

# Testes (Item b):
print(diferenca_listas([1, 2, 3, 4, 5], [2, 4, 6])) # Esperado: [1, 3, 5]
print(diferenca_listas([10, 20], [10, 20, 30]))    # Esperado: []
print(diferenca_listas([1, 2, 3], []))              # Esperado: [1, 2, 3]

```

Q5. Escreva uma função em Python para contar a frequência de um caractere específico em uma string.

- A função deve receber por argumento uma string e um caractere (uma string de tamanho 1), e retornar o número de vezes que esse caractere aparece na string dada.
- Teste a sua função de acordo com as entradas e saídas esperadas a seguir:
 - Entrada:** `conta_caractere("programacao", "a")` → **Saída Esperada:** 3
 - Entrada:** `conta_caractere("Computacao", "z")` → **Saída Esperada:** 0
 - Entrada:** `conta_caractere("", "a")` → **Saída Esperada:** 0

```

# Resposta Questão 5:
def conta_caractere(texto, caractere):
    contador = 0
    i = 0
    while i < len(texto):
        if texto[i] == caractere:
            contador += 1
        i += 1
    return contador

# Testes (Item b):
print(conta_caractere("programacao", "a")) # Esperado: 3
print(conta_caractere("Computacao", "z"))  # Esperado: 0
print(conta_caractere("", "a"))             # Esperado: 0

```

Q6. Escreva uma função em Python que verifica se uma palavra ou frase é um palíndromo (lê-se da mesma forma de trás para frente e de frente para trás, ignorando maiúsculas, minúsculas e espaços).

- A função deve receber por argumento uma string e retornar **True** se a string for um palíndromo e **False** caso contrário. *Dica:* você pode utilizar métodos de string para padronizar as letras e remover espaços.
- Teste a sua função de acordo com as entradas e saídas esperadas a seguir:
 - Entrada:** `eh_palindromo("Arara")` → **Saída Esperada:** True

- **Entrada:** `eh_palindromo("socorram me subi no onibus em marrocos")` → **Saída Esperada:** `True`
- **Entrada:** `eh_palindromo("Python")` → **Saída Esperada:** `False`

Resposta Questão 6:

```
def eh_palindromo(texto):
    # Padroniza letras para minúsculas e remove espaços
    texto_limpo = texto.lower().replace(" ", "")

    # Verifica se a string é igual ao seu inverso
    return texto_limpo == texto_limpo[::-1]

# Testes (Item b):
print(eh_palindromo("Arara")) # Esperado: True
print(eh_palindromo("socorram me subi no onibus em marrocos")) # Esperado: True
print(eh_palindromo("Python")) # Esperado: False
```

Q7. Escreva uma função em Python para formatar e ocultar parte de um endereço de e-mail por motivos de privacidade.

- a) A função deve receber por argumento uma string contendo um e-mail válido (no formato `usuario@dominio.com`). Ela deve retornar o e-mail mascarado, onde apenas os 2 primeiros caracteres do nome do usuário ficam visíveis, substituindo o restante do nome do usuário por 4 asteriscos (`****`) mantendo o domínio intacto.

- b) Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** `mascara_email("joao@gmail.com")` → **Saída Esperada:** `"jo****@gmail.com"`
- **Entrada:** `mascara_email("aluno@ufrj.br")` → **Saída Esperada:** `"al****@ufrj.br"`

Resposta Questão 7:

```
def mascara_email(email):
    partes = email.split("@")
    usuario = partes[0]
    dominio = partes[1]

    usuario_mascarado = usuario[:2] + "****"
    return usuario_mascarado + "@" + dominio

# Testes (Item b):
print(mascara_email("joao.silva@gmail.com")) # Esperado: "jo****@gmail.com"
print(mascara_email("aluno@ufrj.br")) # Esperado: "al****@ufrj.br"
```

Q8. Escreva uma função em Python para extrair apenas as palavras que possuem um comprimento mínimo a partir de uma frase.

- a) A função deve receber dois argumentos: uma string contendo uma frase e um número inteiro k . A função deve retornar uma lista de strings contendo apenas as palavras da frase cujo tamanho seja maior ou igual a k .
- b) Teste a sua função de acordo com as entradas e saídas esperadas a seguir:
- **Entrada:** `palavras_longas("A estrutura de repeticao while e o tipo lista", 5)`
→ **Saída Esperada:** `['estrutura', 'repeticao', 'while', 'lista']`
 - **Entrada:** `palavras_longas("Python e uma linguagem incrível", 7)` → **Saída Esperada:** `['linguagem', 'incrível']`
 - **Entrada:** `palavras_longas("a b c", 2)` → **Saída Esperada:** `[]`

Resposta Questão 8:

```
def palavras_longas(frase, k):  
    palavras = frase.split()  
    resultado = []  
    i = 0  
    while i < len(palavras):  
        if len(palavras[i]) >= k:  
            resultado.append(palavras[i])  
        i += 1  
    return resultado
```

Testes (Item b):

```
print(palavras_longas("A estrutura de repeticao while e o tipo lista", 5))  
# Esperado: ['estrutura', 'repeticao', 'while', 'lista']
```

```
print(palavras_longas("Python e uma linguagem incrível", 7))  
# Esperado: ['linguagem', 'incrível']
```

```
print(palavras_longas("a b c", 2))  
# Esperado: []
```

Q9. Escreva uma função em Python para sanitizar e extrair as palavras-chave (tags) de um texto formatado contendo dados brutos.

- a) A função deve receber por argumento uma string representando uma lista de tags brutas enviadas por um formulário (com espaçamentos extras no início/fim, letras maiúsculas e palavras separadas por hífen). A função deve realizar o tratamento do texto removendo os espaços das extremidades, convertendo todo o texto para minúsculas, substituindo os hífens (-) por espaços () e, por fim, separando as tags por vírgula para retornar uma lista com as palavras-chave tratadas.

Dica: Para resolver esta questão, utilize os métodos de string `.strip()`, `.lower()`, `.replace()` e `.split()`.

- b) Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- Entrada: `processa_tags("PYTHON, MÁQUINA-DE-APRENDER, ESTRUTURA-DE-DADOS ")`
→ Saída Esperada: `['python', 'máquina de aprender', 'estrutura de dados']`
- Entrada: `processa_tags("UFRJ, COMPUTAÇÃO-I, PROVA-FINAL ")` → Saída Esperada: `['ufrj', 'computação i', 'prova final']`
- Entrada: `processa_tags("LÓGICA-DE-PROGRAMAÇÃO ")` → Saída Esperada: `['lógica de programação']`

Resposta Questão 9:

```
def processa_tags(texto_bruto):
    # Utiliza .strip(), .lower(), .replace() e .split()
    texto_sanitizado = texto_bruto.strip().lower().replace("-", " ")
    tags = texto_sanitizado.split(",")
    return [tag.strip() for tag in tags]

# Testes (Item b):
print(processa_tags(" PYTHON, MÁQUINA-DE-APRENDER, ESTRUTURA-DE-DADOS "))
# Esperado: ['python', 'máquina de aprender', 'estrutura de dados']

print(processa_tags(" UFRJ, COMPUTAÇÃO-I, PROVA-FINAL "))
# Esperado: ['ufrj', 'computação i', 'prova final']

print(processa_tags(" LÓGICA-DE-PROGRAMAÇÃO "))
# Esperado: ['lógica de programação']
```